Top #1 Opportunity for Senior Engineers: Agentic Engineering

Time	Subtitle
0s	What's up engineers? Andy Dev Dan here. This is going to be a raw video and it's
4s	more of a message to myself. I just got back from two weeks in Greece completely
8s	unplugged and I used the trip to compact my context around one question. What's
14s	the greatest opportunity available for senior engineers? I came home with one
18s	answer. The opportunity is the same it's been for over a year now. Agentic
24s	engineering. The opportunity is the same, but the size of the opportunity
28s	continues to grow. So we have to grow with it. Andrew Carpathy just called out
33s	aentic engineering directly at the Sequoia AI ascent. When Carpathy named
38s	something, the industry follows, which means the window where you're early is
43s	closing. By the end of 2026, Agentic Engineering will be the default. On this
48s	channel, we've been betting on Agentic Engineering since Clawude Code was
52s	released way back in March 2025. In fact, we've been earlier than that. I
57s	own the domain name agenticengineer.com where myself and thousands of engineers,
1:03	some of the best engineers from top companies that you know have accelerated
1:07	past the industry by pushing the ceiling of agentic engineering, not the floor of
1:13	vibe coding. It's not enough to be early. You have to keep showing up
1:17	because the game is always changing. Here's the fact on the ground. Two
1:21	engineers using the exact same agent with 200k tokens can get massively
1:27	different results. In this video, I want to talk about the five pillars that make
1:32	up the gap between low and high performing agentic engineers. These are
1:36	the key ideas I'm focusing all my time and effort on right now. So, I want to

1:40 share them with you here. We're going to talk about agent harnesses, software

1:44 factories, extensible software, always on agents, and agentic access. If you're

1:50 interested in using these ideas to get the next generation of results today,

1:53 stick around. This is exactly what I'm thinking about every single day to win

1:58 by achieving the ceiling of what's possible. So, let's reset our context on

2:02 the top one opportunity for senior engineers. Agentic engineering, the

2:07 agent harness. Whoever controls the agent harness controls your results. Why

2:12 do I say this? Let's break it down. Let me start with a crazy stat. I'm building

2:16 one new custom agent harness every single day. It's catching up to the

2:20 total number of skills and agents I create. Why is that? It's because I want

2:24 more control over the agentic engineering experience and owning your

2:28 agent harness is exactly how you do that. Let's just be really really raw

2:33 about this. We're talking about cloud code. We're talking about codeex. We're

2:35 talking about open code. These tools are fantastic. They were a great start.

2:39 They're a terrible place to finish. Why is that? It's because the agent is

2:43 everything. It's the agent that gives you the superpowers. The agent unlocks

2:47 the agentic speed, right? You know this human speed is too slow. The agent lives

2:51 in the agent harness and therefore whoever controls the agent harness

2:55 controls your results. These are just the surface level things that you'll

3:00 control when you own your agent harness. As many of you know who tune into the

3:03 channel, I use the piecing agent. The pi coding agent lets me build composable

3:08 units of customization. Okay, it allows me to build a stack of unique

3:13 experiences that combine to build a net new agent harness. So simple examples

3:18 we've talked about on the channel. Multi- aent teams. For those of you that

3:21 are new to this channel, who are just tuning in right now, we don't just talk

3:24 about this stuff. We actually build against it. So let me just open this up

3:27 for you. UIA J team. This is a custom PI agent harness where I've created a

3:32 multi-team orchestration system. So this is a form of multi- aent orchestration,

3:36 right? So I'll say ping your teams. Then the orchestrator will ping all of its

3:40 teams setup brand UI generation validation. And then the team leads also

3:46 connect and coordinate with their workers. So there's three tiers of

3:49 agents here. They're operating in a chat room like interface here. And as you can

3:53 imagine, this is impossible with any out of the box default agent harness that

3:58 you're renting. What's another example of a customization harness? Let me show

4:01 you my default out of the box harness where I've stacked up and composed

4:06 different slices. Here we're pulling out skills, agents, commands from any

4:11 location. Right now it's just using that default since I'm in a temp directory.

4:14 But you can also see here my agent communication system. I have agents

4:18 operating on this network that I can directly prompt and talk to right from

4:22 this agent. Right? This is just one additional example of how you can

4:25 customize and control your agent harness. Whoever owns your agent harness

4:29 controls your results. I'm a huge fan of cloud code. I've been using it since the

4:33 beginning. But this tool is the floor. It's not the ceiling. It's just the

4:38 beginning of what's possible with tools like this. And so I'm building a new

4:41 Asian harness every single day. It's crazy to say that, but it's true. I'm

4:45 experimenting. I'm getting more control over the agentic engineering experience.

4:49 Your tools directly shape what you believe is possible. And with the PI

4:53 agent harness, I see no limits. You can see I've got two agents on this network

4:57 already. I have a presentation Opus 4.7 agent and then I have a helper Gemini

5:02 3.5 flash agent testing out the capabilities of that new model that just

5:05 dropped. And I'm using these two agents together on my agent network through my

5:09 custom agent harness. This is part of the advantage you get when you own your

5:12 agent harness. Right? You want to control the experience of your agentic

5:17 engineering. If you don't own the agent harness, you'll always be limited by

5:20 what another tool is telling you what you can and cannot do. All right? So you

5:23 can create a sandbox tool. Every prompt you write, every thing you do, your

5:28 agent is operating in a sandbox environment just by default. Of course,

5:32 you can recreate sub aent delegation. You can add damage control like we've

5:36 done on the channel in a very custom way. You can add model fallbacks. You

5:40 can route to different models. There's just endless ways to use and build your

5:44 own custom agent harness. And what does this give you in the end? It allows you

5:48 to ultimately build with one tool. You can generate many versions. And again, I

5:52 use the PI agent harness. Shout out to Mario for building such a great tool.

5:56 And there are really kind of two classes of agent harnesses you can build. You

6:00 can build engineering pattern focus agents like the pietoie agent harness we

6:04 talked about last week. I'll link that in the description. You can build agent

6:07 chains. You can build the verifier harness we talked about a few weeks ago

6:10 where you have an agent always checking over the work of another agent. But then

6:13 you can do something really really powerful and more important that many

6:16 engineers are not tapping into. You can build domain specific agents that do one

6:21 thing extraordinarily well. DevOps harness, a testing harness, a billing

6:25 harness. You know, the specialization is truly endless. But if you don't own the

6:30 Asian harness, you cannot specialize your agentic engineering experience. One

6:35 tool, many versions. Specialization is the moat. This will always be true. If

6:40 you can specialize the experience to outperform someone using an out-

6:44 of-the-box agent for your product, for your service, for your application, you

6:47 will win. Whoever controls the agent harness controls your results. So, you

6:51 want to own that. So what's another key pillar of agentic engineering I'm

6:54 centering myself around to get that next set of results. It's of course the

6:58 software factory. And the big idea here is you want to be building factories not

7:03 features. What does this give you? It gives you onspec results every time. So

7:08 whatever you teach your agents to do is what you're going to be able to tap into

7:11 over and over and over. Why are the best engineers using software factories and

7:16 not just building the future themselves? When you are working on your software

7:19 factory, you are building the system that builds the system instead of going

7:23 direct to your new app feature to your new landing page to some new API you're

7:28 building for a client for some service or setting up some accounting

7:32 spreadsheet garbage. Instead of doing any of that yourself, you are building

7:35 the teams of agents, the systems of agents plus code that does it on your

7:40 behalf. And so you're moving the unit of engineering work into the system that

7:45 builds the system. You're moving your focus into the factory. And the first

7:49 place you see this is the plan prompt or the spec prompt. You're giving your

7:54 agent a formula for how you create plans for how engineering work is done. Let's

7:59 be super clear about this. What is a plan? It's a prompt scaled. That's all a

8:03 plan is. It's a more detailed prompt. But that's just the beginning. There's

8:06 planning, there's plan reviewing, there's scouting, there's validating,

8:10 there's the actual building of the thing, there's the testing, there's the

8:13 reviewing. All of this is part of your software factory. And so, who cares? Why

8:17 is the software factory so important? It's because your output per unit of

8:20 time goes parabolic. This is what really matters. I've said this before. I'm

8:24 going to keep saying it. There's an engineer right now that's prompting back

8:26 and forth in their terminal. Maybe they're running two, five, 10 agents.

8:30 The number doesn't really matter because the other engineer, you and I, we are

8:34 going to be building factories. And when you build the factory, you can reproduce

8:39 that focused result over and over and over. You write one prompt and your

8:44 factory of agents and code work together to produce a massive result to produce

8:50 an entire feature to produce an outcome. This frees you from being stuck inside

8:54 the terminal which ties directly into another key pillar AFK agents. We'll get

8:59 to that in a moment. But if this idea makes sense, drop a like and subscribe

9:02 because this means that you understand the true leverage available for agentic

9:08 engineers. You have to invest the time in to your software factory. And this is

9:12 also known as the dark factory. In tactical agent coding, we call these

9:15 ADWs, AI developer workflows. And the whole idea is to combine agents plus

9:20 code to outperform either loan to get repeat results in your system, not

9:25 anyone else's in your system. You teach your agents to build just like you. When

9:29 you build your factories, you template your engineering. It allows you to

9:32 produce that highquality car over and over and over, that high quality result

9:37 over and over and over. Just to kind of put the stamp on this, it's your result.

9:42 You're creating a stamp, a system of reproducible results that's on spec

9:46 every time. Your test, your validation. When you build a factory, it always

9:51 runs. Always getting that verification. You're releasing just like you would.

9:54 Maybe you have a staging environment here that you can preview to validate.

9:58 Maybe you have a whole team of agents that runs and fixes all regressions.

10:02 It's this system, right? It's that factory of results that you're creating

10:07 that you can tap into when you build the software factory. So, this is an idea

10:11 I'm spending a lot of time thinking about. You don't build the feature. This

10:16 is a mindset shift. I'll be honest with you, this is hard to do. You have to

10:19 shift yourself from thinking I'm the engineer that builds the feature. No,

10:22 you're the engineer that builds a system of AI plus code that operates on your

10:28 behalf. You're building the software factory. You are building the system

10:31 that builds the system. This is the key thesis inside of tactical agentic coding

10:35 and on the channel we talk about this all the time because it's so important

10:38 and I want to be like really really blunt with you here. Uh the industry

10:42 right all the top 500 companies everyone is building a software factory. Uh soon

10:47 this will be a requirement. This is how you get massive leverage out of your

10:50 tokens by the end of the year. All of your products that are in production you

10:54 should have software factories. You should be able to write a prompt and

10:56 then see a result near production. And if you're doing really really well and

11:00 you're really owning this, you might be getting close to ZTE, zero touch

11:04 engineering, where you go from prompt directly to production. That's super

11:08 super advanced. We're not going to focus on that here, but that's where this all

11:11 goes. All right, ZTE, zero touch engineering. But none of that matters if

11:15 you don't have the factory that can build the features for you on spec every

11:18 single time. This is not a black or white thing. This is a great thing. It

11:21 takes a lot of practice, a lot of agentic engineering. This is a new skill

11:25 just like full stack, just like backend, just like front end, just like DevOps.

11:28 This is a software engineering skill that takes practice. There's a practice

11:32 in building agents that can operate for you on your behalf, but the end result

11:37 is you get a software factory. Let's move on to the next big idea, extensible

11:41 software. I would say if there are two ideas that I've personally missed, it's

11:46 controlling your agent harness and it's extensible software. being fully honest,

11:51 transparent here. These are the only two ideas I missed in tactical agent coding,

11:54 which is why I've added an extended course into Agent Horizon. And this is

11:58 why I'm talking about um the PI Asian harness so much on the channel right

12:02 now. But it's not just about the harness, right? This is about software

12:06 in general. You want to be writing extensible software. It's very very

12:10 clear why. This is probably not a surprise to you as a senior engineer.

12:13 It's really about the number of changes that's happening in the industry. We

12:16 have model changes. We have new prompts, new tools, new technology. We have

12:21 change happening all the time. And the best way adapt to change is to build it

12:26 into your software. You want to be building pluggable software, easy to

12:30 extend, easy to change, easy to tweak. It needs to be adaptable because, as you

12:35 know, models will change, tools will change. These things are are moving at

12:39 light speed. The pace will only continue. Software is going parabolic

12:42 right now. GitHub is crashing all the time. critical services changing all the

12:47 time because we'll be operating at the agentic speed. And so if your software

12:51 is complex, if your software is slow, if it's brittle, if you are vibe coding

12:55 trash, you have not seen or read, if you're generating more slop, creating

12:59 more tech debt in your technology, you will suffer, right? Extensible software

13:05 is something I'm thinking about a lot right now. And again, there's two kind

13:08 of classes for this. There's engineering work and there's product work. There's

13:12 enhancing your development. And a great example of this is your agent harness.

13:16 And then there is production work. Whether you're deploying agents into

13:20 production for agentic products or if you have traditional software where you

13:24 know AI isn't involved, it doesn't really matter. You need adaptation.

13:28 Agent harness is a great example of this, right? Constantly. There are

13:31 different tools, there are new prompts, there's new agents you want to be

13:34 testing, different system prompts. You want to be able to control the model

13:37 that you're using all the time. This is why the PI agent harness is so

13:40 important. It's so swappable. It's so dynamic. You can change this thing on

13:44 the fly. When you're thinking about extending your lead as an agentic

13:47 engineer, I highly, highly recommend you think about how you can make all of your

13:50 software more extensible. Pluggability is the key. Extensibility is the key.

13:56 Composability is the key. If your software has a million trillion rules

14:00 and operates in a very specific line of cascading if statements, the next year

14:06 is going to be really, really hard for you because your agents will be really

14:08 slow. they'll be making a lot of mistakes and you have to teach them in

14:11 your software how to navigate all that. If you have extensible software,

14:14 adaptable software, if you are adding and not modifying, you will have a

14:19 massive advantage. This is one of my favorite software principles, open to

14:22 extension, close to modification. Right. So, I'm thinking about that a lot as I

14:26 use agents every single day as I'm building both a new agentic engineering

14:30 workflows for myself to improve and increase my speed, but also on the

14:35 product side, thinking about how to build great products that survive and

14:38 thrive in a world where change is the default mode. Build extensible software.

14:44 All right, makes sense. And if you're a senior engineer, you know this. These

14:47 are ideas I'm re-emphasizing as I'm resetting my context for this next phase

14:51 of work. Always on agents. When we talk about the ceiling of agentic

14:54 engineering, this is one of the most important ideas. It's not enough to just

14:58 have an agent that's always running. Anyone can set up a crown job, spin up

15:01 an agent, run a wild loop on an agent, and just have it always be burning

15:06 tokens. The key here is useful tokens. In Silicon Valley, in a lot of these

15:10 tech companies, there's a term called token maxing. You've probably heard of

15:14 it. Token maxing sits at the lowest level of our economic funnel of

15:18 tokconomics. Okay, so there's three levels here. First, you need to use more

15:23 tokens. Then, you need to make your tokens useful. And then you capture the

15:26 revenue generated by the value you created. Okay? So, these are the three

15:30 levels. Now, if you're token maxing, you're very likely down here. You're

15:34 just using more tokens. This is great. It's a great place to start. Terrible

15:37 place to finish. Once you're spending tokens in your agentic systems, the next

15:41 step is to make them valuable. And I think this is, you know, where a lot of

15:46 companies, a lot of engineering teams, a lot of adopters of agents are really

15:50 sitting right now. A lot of tokens getting generated, not a lot of value

15:53 getting generated. And this is the trick of engineering. Only after you do that

15:56 are you able to move to that highest level. Now, why am I saying this? Why is

15:59 this important? It's because once you generate valuable tokens and once you

16:03 can capture that arbitrage, the next step, the only clear step is to turn

16:07 them on all the time. This is the ceiling of agentic engineering. Agents

16:11 that are on for you, operating, working while you sleep. But in order to get

16:16 there, you must understand the token arbitrage. This is what everyone with

16:20 their head on their shoulders is trying to do. If you're a senior engineer

16:23 looking at the best way to spend your time, it's following this equation.

16:27 We're all purchasing tokens right now. If you are at level two, you've made

16:30 them useful, then you roll them into your products or your engineering

16:34 experience. Once you've generated value for your users or for yourself, the next

16:39 thing to do is to capture the value, right? You need to capture the actual

16:42 revenue generated by your tokens. And so, it's these three levels that matter.

16:47 First you use more tokens. Then you make your tokens valuable. And then from your

16:50 tokens that are valuable, you need to turn that into real revenue. That's the

16:54 arbitrage that's happening right now. This is what everyone is and should be

16:58 focused on right now. You buy the tokens at a certain price. You make them

17:01 useful. You roll them into your products and your engineering experiences. And

17:05 then you need to capture the actual revenue generated from them. This is the

17:08 token arbitrage. This is what matters. This is tokconomics. And then the whole

17:12 key is once you can successfully do this, you turn it on. You have always on

17:17 agents, AFK agents. When you do this right, you then have a new KPI, right?

17:21 Your rising API bill is a new productivity KPI, but only after you get

17:26 out of level one and level two, because then this turns into a classic business

17:29 problem. How many tokens can you spend? What are they worth to the business? And

17:33 then you capture that difference. So, if you pay a dollar for a token and you can

17:37 through your business generate a $110 worth of value for that token and then

17:42 you capture that 10 cents, right, you have an infinite cash generating glitch,

17:47 also just known as a business. And then all you need to do after that, just like

17:51 when you have a great ad campaign that's running, you just scale it to the moon,

17:55 right? You spend as many tokens as possible. This is literally what you see

17:59 in the big AI labs. With more tokens that they can sell, they make more

18:03 money. But contrary to popular belief, I believe this goes all the way through

18:06 the stack, all the way down to uh companies, all the way down to

18:09 individual developers like you and I. If you can buy a token for a dollar, run it

18:13 through your business process and sell it for $2, that's a 2x right there. And

18:17 then all you need to do is smash tokens and scale it up. So it looks something

18:21 like this. Your API cost should go up and the value you're generating should

18:24 go up directly with it. Ideally greater than the cost. It doesn't need to be a

18:29 linear scale. It most likely will be, but this is a simple idea. And once you

18:32 do this, the whole idea is to scale it, right? Turn that system, turn that

18:36 agent, turn that product on all the time. And so I'm thinking about this and

18:40 the trick here is they have to be useful tokens. Token maxing once again, it's

18:44 the floor. I can spend millions of tokens right now. I think you would be

18:48 very very surprised to see my token usage is actually quite low. My token

18:52 growth is a very very smooth curve because once I find that key value, that

18:57 arbitrage, once I unlock value generation and turn that into revenue

19:02 capture for whatever business, products, clients I'm working with, that's when

19:05 you scale it to the moon. But it's only at level three. If you're doing it at

19:09 level one or level two, you're just token maxing. Once you do that, you turn

19:12 that agent always on, you turn that product, that system always on, that

19:15 agent harness, turn it on. But always think through that first. And this is

19:19 the key difference between that high performing engineer using the exact same

19:23 agent and the exact same number of tokens. If you're generating more value

19:27 and you're capturing the value from the tokens you generate, you will be ahead

19:30 of the pack. You will be capturing more revenue and therefore delivering more

19:34 value. And only then do you turn it into an always on agent. There are a million

19:39 agent crown jobs running. 90% of them are dead useless and just burning cash,

19:43 burning tokens. Agentic access. So the headliner here is simple. You likely

19:47 already understand this idea, but it's very important to emphasize because I

19:52 know for a fact that you, the senior engineer, you probably haven't given

19:56 your agent API access to all the things you really could to maximize your

20:01 agentic speed. API access is a requirement of agentic speed. Let's just

20:05 quickly go through this. The ideas here are simple. You already understand this,

20:07 but it's really important to re-emphasize. As I'm refocusing my

20:11 personal context for this next phase of work, I'm thinking about always moving

20:15 at the agentic speed. If there's something an agent can do, uh you have

20:19 to really deeply ask yourself why it isn't it. And a lot of the times it's

20:23 just because you haven't given your agent the API access it needs. But this

20:27 is how you get the agentic speed. Okay? And this is the only way to operate now.

20:31 CLI tools, rest, web hooks, RPC clients, you know the deal. Agents only command

20:35 what they can programmatically reach. And in order for them to act and operate

20:39 like you, you need to give them the tools that you have access to. There are

20:42 limits here. You don't want to be giving production access so that they can nuke

20:46 your production databases, your volumes, your resources. As talked about in a

20:49 recent video, you want to lock down that bash tool so that your agents never do

20:53 anything catastrophic. But the idea is very simple. You want to avoid the token

20:57 tax. So what is the token tax? It's basically anything your agent does that

21:01 is unnecessary strictly because you haven't given it direct API access. You

21:06 want your agents using tokens only when they need to to do the thing that is the

21:10 most useful and valuable to you, your company, your product, your users. Don't

21:14 pay a token tax. And this is that simple mindset shift that you already know. I

21:18 know that you know this API access is a requirement of moving at the agentic

21:22 speed. Agents, they're 10, hundreds, thousands of times faster at operating

21:26 digital information than you or I. This is also why we don't work on the

21:29 feature. We work on the agents that build the feature, right? We build the

21:32 system that builds the system. It's the same thing. We want to be moving at the

21:35 agentic speed, the speed of agents, but you can't do that if they don't have API

21:40 access. So once again, I am refocusing my context around this. There are things

21:44 you and I are doing right now. We don't have to be, but we are because we just

21:48 haven't invested the time to give our agents the tools, the access, the

21:52 resources that they need. So make sure you do that exactly. Okay. So these are

21:56 the five pillars that are going to compound your advantage toward the one

22:00 opportunity that matters the most right now. Okay. It's agentic engineering.

22:03 Karpathy said it. The entire industry is catching up with this. You're going to

22:06 hear these kind of similar terms come around all the time, right? Dark

22:10 factory, software factory, air developer workflows, always on agents, AFK agents,

22:15 agentic access, extensibility, agent harnessing, right? Harness engineering.

22:20 These are all ideas that are going to come up over and over and over in

22:23 different forms and shapes. Uh, as I've been saying on the channel, success is

22:27 often about doing a few very simple things over and over and over and over

22:32 and over. And and you've seen this, you know this that the name of the game, the

22:35 one opportunity, the thing you need to focus on, and I'm saying you, but

22:39 really, as I mentioned, I created this presentation. A lot of the work I do is

22:43 like a note to myself. It's a reminder to myself. You want to be agentic

22:46 engineering. You want to be building the system that build the system. You want

22:49 to be focusing on pillars that compound, actions that compound so that you can

22:54 really tap into the ceiling of agentic engineering. Stack your advantage,

22:57 compound the opportunity, raise the ceiling of your agentic engineering.

23:01 Vibe coding is the lowest hanging fruit. Do not sit in the terminal prompting out

23:06 your features. Build the software factory. Own the agent harness. Use

23:10 extensible products like the PI agent harness, but also make your products

23:14 extensible by nature. Build it into the fabric of the work you're doing. Learn

23:19 to arbitrage your tokens. Spending a bunch of tokens that do not generate

23:23 value is meaningless. You need to spend more tokens. You need to figure out

23:26 which ones are actually valuable. And then you need to capture the value

23:30 usually via revenue from your tokens. Then once you do that, you turn that

23:34 thing on so it runs 24/7. Always on agents, AFK agents, get that maximum

23:40 value you can out of that system. And then, you know, more of a reminder than

23:43 anything, if there's something you're doing that your agents can't do, why

23:47 aren't you, right? Give your agents access. Make sure you're always focused

23:52 on agentic access. Agent first systems, agent first products, agent first

24:00 workflows. Expose your CLIs and APIs everywhere. Code bases, products,

24:05 devices, tools. Agents can only command what they can reach. If you're not doing

24:08 this, but you're still using agents, you're paying a token tax. You're

24:12 burning up tokens because you didn't put the upfront investment into the system

24:17 that you need to. Again, this is more of a reminder to myself. A lot of the

24:20 things I do on the channel, they're for me as much as they are for you. But this

24:25 is what I'm focusing on every day as an engineer right now. The gap between, you

24:28 know, the top 2% of engineers and everyone else is widening every single

24:32 week. Why is that? It's because they are compounding their advantage with this

24:36 one opportunity. It's very rare that you're presented with an opportunity to

24:40 compound advantage. Don't miss this. Move slow now to move fast later. Invest

24:45 in your agentic layer. Don't sit in the terminal vibe coding. Agentic engineer

24:49 your work. build the system that builds the system. I have a bunch of links that

24:53 I'll share in the description for you. Previous videos where we talk about the

24:57 pietoie customized agent harness. I'll of course share tactical agentic coding

25:02 for anyone that's interested in understanding what is possible right now

25:06 with agentic engineering and what will be possible once you put the reps in. A

25:10 big shout out to uh Mario and everyone working on the PI Asian harness. This

25:14 tool has been incredible to work with. As I mentioned, I'm building a new Asian

25:17 harness every single day now, which is really absurd. You can imagine I have a

25:23 software factory that lets me do this. So, bunch of links in the description

25:26 for you. Check all them out. Bunch of free stuff, some paid stuff if you're

25:30 interested in that extra push. These are the five big ideas I'm focusing on. I'm

25:34 reentering myself on. I'm emphasizing in all of my work, and so I recommend you

25:38 do the same. Comment down below if you think I missed anything. You'll notice

25:41 here I didn't mention models a single time. That's because models matter less

25:46 and less. They matter for bleeding edge work, but for a lot of the work, 80 90%

25:50 of the work we're doing on a daily basis, what matters is the systems you

25:53 place around your agents, we are talking about agentic engineering, which to be

25:58 clear, it's the process of engineering with intelligence that can operate on

26:02 your behalf. So, five big ideas for you. I've got more resources linked for you

26:06 in the description. Shout out Pi. Shout out Mario. Uh, shout out Andrew

26:09 Carpathy. Shout out Claw Code. And um, thank you for watching to the

26:15 end here. You know where to find me every single week. Stay focused and keep

26:19 building.